Writing Better Requirement Specifications.pdf
PDF

extract all the pdf correctly from it and arrange it

Got it! I've extracted the content from your PDF and organized it into a clean, copy-paste-friendly outline. I preserved every point and arranged it by sections, with clear headers and bullet lists so you can skim or reuse easily.

Writing Better Requirement Spec…

Writing Better Requirement Specifications — From Poor to Powerful Requirements

(Software Requirements Engineering, May 20, 2025)

Table of Contents

---

1) Example 1 — Ambiguous Functional Requirement

1.1 Bad Example

> "The system shall provide fast search functionality with good results."

1.2 Why it's Bad

- **Ambiguity:** "fast" is undefined.
- **Unmeasurable:** No way to test "fast enough."
- **Vague criteria:** "Good results" is subjective.
- **Incomplete:** Doesn't specify searchable content or ranking.
- **Unverifiable:** No objective acceptance test.

1.3 Better Version

Users will:

- **Response time:** Receive search results within **3 seconds** after entering up to **5 keywords**.
- **Result display:** See the **top 20 matches** in a ranked list.
- **Ranking algorithm:** Results prioritized by matches in **product titles (primary)** and **descriptions (secondary)**.
- **Performance at scale:** Consistent performance when searching across **100,000 products**.

---

2) Example 2 — Vague Non-functional Requirement

2.1 Bad Example

> "The system must have good uptime."

2.2 Why it's Bad

- **Subjective:** "good" is undefined (90%? 99.9%?).
- **No measurement period:** Over what timeframe?
- **No exclusions:** Maintenance counting unclear.
- **No recovery metrics:** Recovery expectations missing.
- **Not testable:** No verification method.

2.3 Better Version

Users will:

- **Availability guarantee:** Access the application **99.5%** of the time **each month**.
- **Maintenance policy:** Scheduled maintenance **≤ 4 hours/month**, announced **5 days in advance**, **excluded** from availability metrics.
- **Recovery time:** Regain access to **all core functions** within **30 minutes** after any unexpected outage.

---

3) Example 3 — Unmeasurable Usability Requirement

3.1 Bad Example

> “The user interface must be user-friendly.”

3.2 Why it’s Bad

- **Subjective term:** “User-friendly” varies by person.
- **No target users:** Which user groups?
- **No metrics:** No measure of “friendliness.”
- **No specifics:** Which usability aspects matter?
- **Unverifiable:** No clear “done” definition.

3.3 Better Version

First-time users with no prior training will:

- **Task efficiency:** Complete order submission in **< 5 minutes**.
- **Error tolerance:** Make **≤ 2 errors** during order submission.
- **Satisfaction:** Rate the interface **‘easy’ or ‘very easy’** (**≥ 85%** on a 5-point scale).

---

4) Example 4 — Incomplete Security Requirement

4.1 Bad Example

> “The system shall be secure.”

4.2 Why it’s Bad

- **Too general:** Which security dimensions?
- **No standards:** No compliance references.
- **No threat model:** Which threats to mitigate?
- **No auth details:** Missing authentication rules.
- **No verification:** How to test “secure enough”?

4.3 Better Version

Users will:

- **Secure communications:** All data via **HTTPS** with **TLS 1.3**.

- **Data protection:** Personal data stored with **AES-256** encryption.
- **Auth security:** Passwords **≥ 8 chars**, include **numbers + special characters**; temporary **lockout after 5 failed attempts**.
- **Payment security:** Payments through a **PCI-DSS compliant** interface.

---

5) Example 5 — Technically Infeasible Requirement

5.1 Bad Example

> "The system must be 100% error-free and never experience downtime."

5.2 Why it's Bad

- **Unrealistic:** Zero errors / 100% uptime is impossible.
- **Unfeasible:** Hardware, network realities.
- **Economically impractical:** Near-perfect reliability is exponentially costly.
- **Ignores trade-offs:** Cost vs reliability not recognized.
- **Misleading expectations:** Stakeholders get the wrong picture.

5.3 Better Version

Users will:

- **System availability:** Reliable access during **8am–8pm** with **99.9%** availability.
- **Error rate:** Critical errors **< 1 in 1,000 transactions**.
- **Error handling:** Users notified; issues resolved within **1 business hour**; **proactive monitoring** in place.

---

6) Example 6 — Solution Constraint Instead of Requirement

6.1 Bad Example

> "The system shall use a MySQL database and PHP for the backend."

6.2 Why it's Bad

- **Dictates implementation:** Says **how**, not **what**.
- **Limits innovation:** Prevents better tech choices.
- **Not a requirement:** It's a design decision, not a user/business need.
- **Obsolescence risk:** Tech changes faster than requirements.
- **Means over ends:** Doesn't state the needed outcome.

6.3 Better Version

Users will:

- **Data access performance:**
  - Access individual customer records in **< 0.5 seconds**.
  - Generate reports up to **10,000 records** in **< 3 seconds**.

---

7) Example 7 — Multiple Requirements in One

7.1 Bad Example

> "The system shall allow users to manage their profile, send messages to other users, upload documents, and generate reports while maintaining privacy and security."

7.2 Why it's Bad

- **Not atomic:** At least **5** separate requirements bundled.
- **Hard to trace:** Poor upstream/downstream traceability.
- **All-or-nothing:** Can't prioritize or partially implement.
- **Difficult testing:** No per-requirement verification.
- **Ambiguous responsibility:** Unclear ownership.

7.3 Better Version

Split into separate, uniquely identifiable requirements:

- **REQ-1.1:** Users view & edit profile in a **dedicated profile section**.
- **REQ-1.2:** Users send **private messages** via an **integrated messaging UI**.
- **REQ-1.3:** Users upload **PDF, DOC, DOCX** via **file management**.
- **REQ-1.4:** Users generate **activity reports** for **custom date ranges** in **reporting dashboard**.
- **REQ-1.5:** Users control privacy with **granular visibility permissions**.

---

8) Characteristics of Good Requirements (CORRECT)

8.1 Part 1

- **Clear & Unambiguous:** Avoid vague terms (“fast,” “user-friendly,” “secure”).
- **Complete:** Inputs, outputs, conditions clearly defined; no gaps.
- **Verifiable & Testable:** Objective validation with measurable criteria.
- **Consistent:** No contradictions across requirements.

8.2 Part 2

- **Atomic:** One thing per requirement; independently implementable & testable.
- **Traceable:** Map to business needs (up) and design/code/tests (down).
- **Feasible:** Achievable with available tech/time/resources.
- **Necessary:** Addresses a real stakeholder need; not redundant.

8.3 Part 3

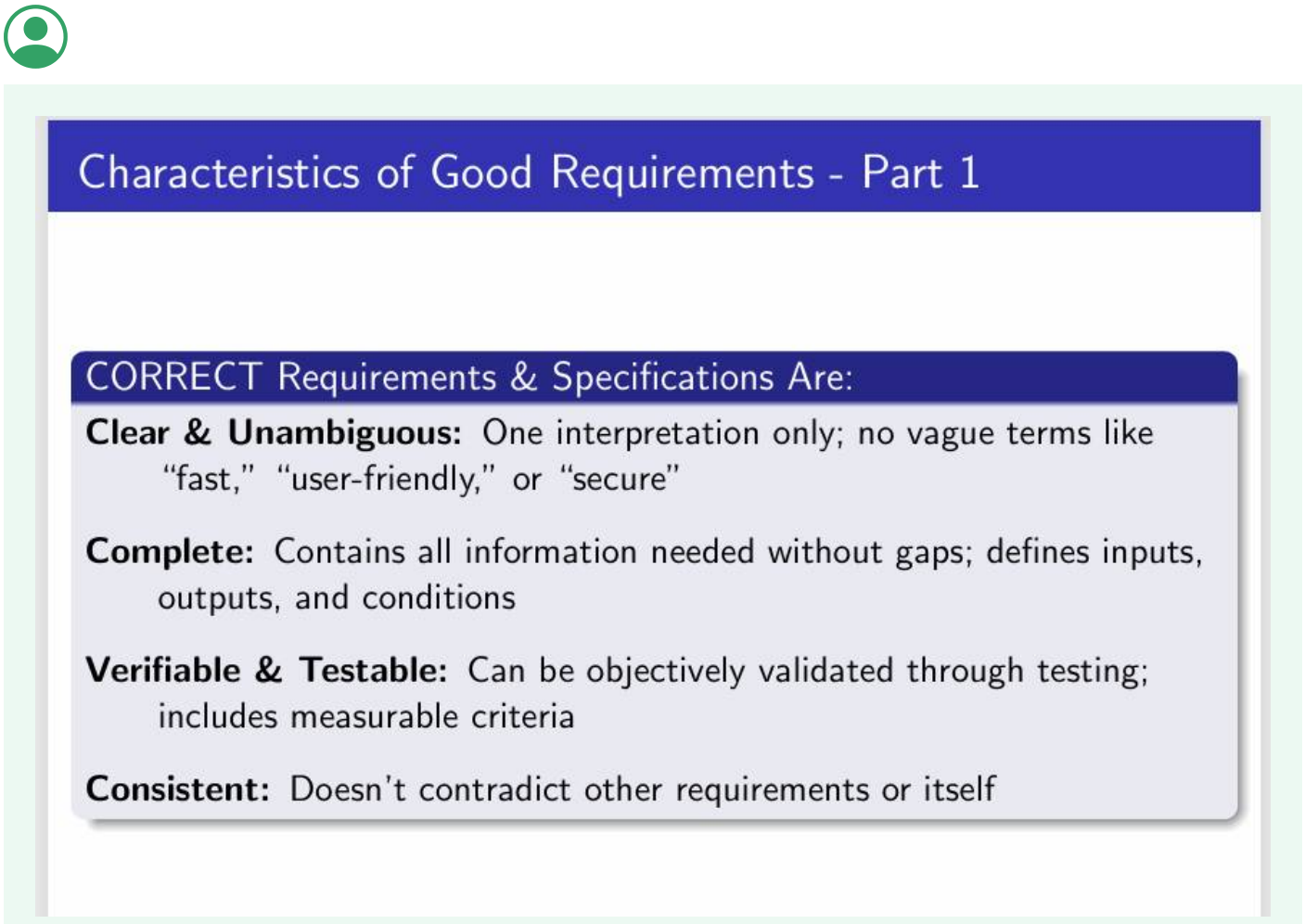
- **Implementation-free:** Focus on **WHAT**, not **HOW** (unless legitimate constraints).
- **Quantifiable:** Prefer metrics (time, %, counts) over qualitative terms.
- **User-focused:** Describe from the **user’s perspective**.
- **Prioritized:** Indicate relative importance for planning.

Bonus: Quick “Bad → Better” Cheat Sheet (for your SPL/DevFiesta docs)

- **“Fast search”** → *“≤3s for up to 5 keywords; top-20 ranked; title/description weights; scales to 100k items.”*
- **“Good uptime”** → *“99.5%/month; ≤4h scheduled maintenance with 5-day notice; ≤30m recovery.”*
- **“User-friendly UI”** → *“<5m first-time task; ≤2 errors; ≥85% rate ‘easy/very easy’.”*
- **“Be secure”** → *“TLS 1.3; AES-256 at rest; password policy + lockout; PCI-DSS payments.”*
- **“Never down, no errors”** → *“99.9% (8–20h); <0.1% critical error rate; 1-hour resolution; monitoring.”*
- **“Use MySQL+PHP”** → *“Record <0.5s; 10k-row report <3s.”*
- **Bundled requirement** → **Split into REQ-x.y** with one atomic outcome each.

If you want, I can also export this as a neatly formatted **DOCX/PDF** with page headers, or convert to a **requirements template** (MoSCoW, IEEE 29148-style, or your own SPL schema).

Sources



Characteristics of Good Requirements - Part 2

CORRECT Requirements & Specifications Are:

Atomic: Addresses one thing only; can be independently implemented and validated

Traceable: Links to business needs upward and to design/code/tests downward

Feasible: Can be implemented with available technology, time, and resources

Necessary: Fulfills an actual stakeholder need; not redundant

Characteristics of Good Requirements - Part 3

CORRECT Requirements & Specifications Are:

Implementation-free: Focuses on WHAT is needed, not HOW to build it (unless constraints are legitimate)

Quantifiable: Uses specific metrics where possible (time, %, counts) instead of qualitative terms

User-focused: Describes functions from the user's perspective rather than system perspective

Prioritized: Indicates relative importance to guide development decisions

explain these parts properly and give at least 2-3 example for each that doest foloow the criteria and give the correct version then

Perfect — you've shared the three slides on **Characteristics of Good Requirements (Part 1–3)**. Let me now explain them in detail and illustrate each criterion with **bad examples (that fail the rule)** and **improved correct versions**.

🟦 Part 1 — Core Qualities

1. Clear & Unambiguous

- **Meaning:** A requirement should have only one interpretation, leaving no room for subjective words.
- **Bad Example 1:**

  "The system should be fast."
  ❌ Ambiguous — "fast" could mean 1s, 10s, or "fast compared to what?"

  - **Correct Version:**

    "The system shall return search results within **3 seconds** for queries of up to **5 keywords** on a database of **100,000 records**."

- **Bad Example 2:**

  "The interface must be user-friendly."
  ❌ "User-friendly" varies for different users.

  - **Correct Version:**

    "First-time users shall complete order placement in **< 5 minutes** with **≤ 2 errors**."

---

2. Complete

- **Meaning:** Requirement must include all inputs, outputs, and conditions — no missing details.
- **Bad Example 1:**

  "The system should generate monthly reports."
  ❌ What kind of reports? For whom? What data?

- **Correct Version:**

  > "The system shall generate **monthly sales reports** including **total revenue, number of orders, and top 10 products**, available in **PDF and Excel formats**, delivered to the **sales manager's dashboard**."

- **Bad Example 2:**

  > "The app must notify users."
  > ❌ Notify for what events? Email or SMS?

- **Correct Version:**

  > "The app shall send **SMS notifications** to users **5 minutes before delivery arrival**."

---

3. Verifiable & Testable

- **Meaning:** There must be measurable, objective criteria for testing.
- **Bad Example 1:**

  > "The website should load quickly."
  > ❌ No clear benchmark for testing.

- **Correct Version:**

  > "The homepage shall load fully in **< 2 seconds** on a **3G connection**."

- **Bad Example 2:**

  > "The system should be reliable."
  > ❌ Vague and untestable.

- **Correct Version:**

  > "The system shall maintain **99.5% uptime per month**, excluding scheduled maintenance."

---

4. Consistent

- **Meaning:** A requirement must not contradict other requirements.
- **Bad Example 1:**

  "The password must be exactly 6 characters."
  (Elsewhere: "Password must be at least 8 characters.")
  ❌ Contradiction between requirements.

  - **Correct Version:**

    "Passwords must contain **at least 8 characters** with a mix of **letters, numbers, and symbols**."

- **Bad Example 2:**

  "Reports must be downloadable in PDF only."
  (Elsewhere: "Reports must be downloadable in Excel format.")
  ❌ Inconsistent output formats.

  - **Correct Version:**

    "Reports must be downloadable in **both PDF and Excel formats**."

🟦 Part 2 — Structural Qualities

5. Atomic

- **Meaning:** Each requirement must describe one single feature.
- **Bad Example 1:**

  "The system shall allow users to update profiles, send messages, and upload files."
  ❌ Bundled three features in one requirement.

  - **Correct Version:**
    - "REQ-1: Users shall update their profile."
    - "REQ-2: Users shall send messages."
    - "REQ-3: Users shall upload files."
- **Bad Example 2:**

"The user shall login and reset their password."
❌ Two distinct functions.

- **Correct Version:**
  - "REQ-1: User shall log in with credentials."
  - "REQ-2: User shall reset their password via email OTP."

---

6. Traceable

- **Meaning:** Each requirement must link to a business goal and to design/code/tests.
- **Bad Example 1:**

  "Add dark mode."
  ❌ No clear business or stakeholder traceability.

- **Correct Version:**

  "REQ-7: The system shall provide a dark mode **to reduce eye strain for medical staff working night shifts**, aligning with business goal **BG-3: Improve usability for staff."

- **Bad Example 2:**

  "Allow guest checkout."
  ❌ No linkage to strategy.

- **Correct Version:**

  "REQ-9: Guest checkout shall be enabled **to reduce cart abandonment**, linked to business metric **increase conversion rate by 10%**."

---

7. Feasible

- **Meaning:** Requirement must be achievable with existing tech/time/budget.
- **Bad Example 1:**

"The app must respond instantly to all queries."
❌ "Instantly" is technically infeasible.

- **Correct Version:**

  "The app shall respond within **2 seconds** for **95% of queries** under normal load."

- **Bad Example 2:**

  "The system shall predict customer behavior with 100% accuracy."
  ❌ Impossible for ML/AI.

- **Correct Version:**

  "The system shall predict customer purchase behavior with an **accuracy of at least 85% (F1 score ≥ 0.85)."

---

8. Necessary

- **Meaning:** Requirement should serve a real need, not redundancy.
- **Bad Example 1:**

  "The app shall play background music on every screen."
  ❌ Nice-to-have, but not necessary for business.

- **Correct Version:**

  (Removed if not justified by user/stakeholder need).

- **Bad Example 2:**

  "The app shall support fax communication."
  ❌ Outdated and irrelevant.

- **Correct Version:**

  "The app shall support **email and push notifications**, which fulfill current stakeholder communication needs."

🟦 Part 3 — Practical Qualities

9. Implementation-free

- **Meaning:** Define **what**, not **how**, unless the "how" is a valid constraint.
- **Bad Example 1:**

  "The backend must be developed in PHP and MySQL."
  ❌ Dictates implementation.

  - **Correct Version:**

    "The system shall support **10,000 concurrent users** and generate reports in **under 5 seconds**."

- **Bad Example 2:**

  "The app must use Bootstrap for design."
  ❌ Tool choice, not requirement.

  - **Correct Version:**

    "The app shall provide a **responsive UI accessible on desktop, tablet, and mobile devices.**"

---

10. Quantifiable

- **Meaning:** Use measurable numbers, not vague adjectives.
- **Bad Example 1:**

  "The website should be easy to navigate."
  ❌ Subjective.

  - **Correct Version:**

    "New users shall complete **registration in ≤ 3 minutes** with **≤ 1 error.**"

- **Bad Example 2:**

  "The system should support many users."
  ❌ Undefined "many."

  - **Correct Version:**

> "The system shall support **5,000 concurrent users** with response time ≤ **2 seconds**."

---

11. User-focused

- **Meaning:** Requirements should describe what the **user gets**, not what the system does internally.
- **Bad Example 1:**

  > "The database shall encrypt data."
  > ❌ System-centric, not user perspective.

  - **Correct Version:**

    > "Users' personal data shall remain confidential and accessible only to authorized accounts."

- **Bad Example 2:**

  > "The system shall log errors."
  > ❌ Internal task.

  - **Correct Version:**

    > "Users shall be notified with a clear error message if an action fails."

---

12. Prioritized

- **Meaning:** Requirements should be ranked to guide decision-making.
- **Bad Example 1:**

  > "The system shall have live chat, multilingual support, AI recommendations, and social sharing."
  > ❌ No prioritization, all lumped together.

  - **Correct Version:**

    - **Must-have:** Multilingual support for English & Bangla.
    - **Should-have:** Live chat support.

- **Could-have:** AI recommendations.
- **Won't-have (this release):** Social sharing.

- **Bad Example 2:**

  "All features are critical."
  ❌ Unrealistic, blocks planning.

- **Correct Version:**

  Categorized by **MoSCoW prioritization** (Must, Should, Could, Won't).

✅ With these 12 criteria (CORRECT model), your requirements move from **ambiguous → precise, testable, and valuable.**

Would you like me to **put all these corrected examples into a structured IEEE-style requirements template** (with ID, description, rationale, acceptance criteria) so you can directly reuse it for your SPL project?